



INSIYA ZAIDI

✦ 23BEC073 ✦

BTECH(ECE -IV SEM)



1) PROGRAM – CONVERSION FROM DECIMAL TO BINARY AND VICE-VERSA

Decimal to binary conversion

```
#include <iostream>

#include <string>

using namespace std;

string decimalToBinary(int decimal) {

    string binary = "";

    if (decimal == 0)

        return "0";

    while (decimal > 0) {

        binary = to_string(decimal % 2) + binary;

        decimal /= 2;

    }

    return binary;

}

int main() {

    int decimal;

    cout << "Enter a decimal number: ";

    cin >> decimal;

    if (decimal < 0) {

        cout << "Please enter a non-negative integer.\n";

    }

    else {

        string binary = decimalToBinary(decimal);

        cout << "Binary representation: " << binary << endl;

    }

    return 0;

}
```

OUTPUT -

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\LENOVO\Desktop\Insiya\.vscode> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ exp1.cpp -o exp1 } ; if ($?) { .\exp1 }
Enter a decimal number: 13
Binary representation: 1101
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> []
```

Binary to decimal conversion

```
#include <iostream>

#include <string>

#include <cmath>

using namespace std;

int binaryToDecimal(string binary) {

    int decimal = 0;

    int length = binary.length();

    for (int i = 0; i < length; i++) {

        char bit = binary[i];

        if (bit == '1') {

            decimal += pow(2, length - i - 1);

        } else if (bit != '0') {

            cout << "Invalid binary digit detected: " << bit << endl;

            return -1;

        }

    }

    return decimal;

}

int main() {

    string binary;

    cout << "Enter a binary number: ";

    cin >> binary;

    int decimal = binaryToDecimal(binary);

    if (decimal != -1) {

        cout << "Decimal representation: " << decimal << endl;

    }

    return 0;

}
```

OUTPUT -

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ exp1.cpp -o exp1 } ; if ($?) { .\exp1 }
● Enter a binary number: 1011
  Decimal representation: 11
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```

2)PROGRAM TO GENERATE FIBONACCI SEQUENCE

```
#include <iostream>

using namespace std;

int main() {

    int n;

    cout << "Enter the number of terms in the Fibonacci sequence: ";

    cin >> n;

    if (n <= 0) {

        cout << "Please enter a positive integer greater than 0." << endl;

        return 0;

    }

    int first = 0, second = 1;

    cout << "Fibonacci Sequence: ";

    if (n == 1) {

        cout << first << endl;

    }

    else {

        cout << first << " " << second << " ";

        for (int i = 3; i <= n; i++) {

            int next = first + second;

            cout << next << " ";

            first = second;

            second = next;

        }

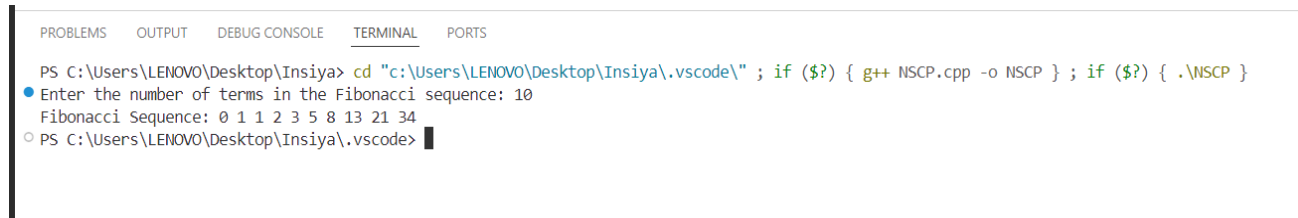
        cout << endl;

    }

    return 0;

}
```

OUTPUT –



3)PROGRAM TO COMPUTE SUM OF SINE AND COSINE SERIES

```
#include <iostream>

#include <cmath>

using namespace std;

double toRadians(double degree) {

    return degree * M_PI / 180.0;

}

long long factorial(int n) {

    long long fact = 1;

    for (int i = 2; i <= n; i++)

        fact *= i;

    return fact;

}

double sine(double x, int terms) {

    double sum = 0;

    for (int i = 0; i < terms; i++) {

        double term = pow(-1, i) * pow(x, 2 * i + 1) / factorial(2 * i + 1);

        sum += term;

    }

    return sum;

}

double cosine(double x, int terms) {

    double sum = 0;

    for (int i = 0; i < terms; i++) {

        double term = pow(-1, i) * pow(x, 2 * i) / factorial(2 * i);

        sum += term;

    }

    return sum;

}

int main() {

    double degree;

    int terms;

    cout << "Enter the angle in degrees: ";
```

```
cin >> degree;

cout << "Enter the number of terms: ";

cin >> terms;

double radians = toRadians(degree);

double sin_sum = sine(radians, terms);

double cos_sum = cosine(radians, terms);

double total_sum = sin_sum + cos_sum;

cout << "Sum of sine series: " << sin_sum << endl;

cout << "Sum of cosine series: " << cos_sum << endl;

cout << "Sum of both series: " << total_sum << endl;

return 0;

}
```

OUTPUT –

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter the angle in degrees: 45
Enter the number of terms: 7
Sum of sine series: 0.707107
Sum of cosine series: 0.707107
Sum of both series: 1.41421
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```

4)PROGRAM TO COMPUTE SIMPLE INTEREST AND COMPOUND INTEREST

```
#include <iostream>

#include <cmath>

using namespace std;

int main() {

    double principal, rate, time;

    cout << "Enter principal amount: ";

    cin >> principal;

    cout << "Enter annual interest rate (in %): ";

    cin >> rate;

    cout << "Enter time in years: ";

    cin >> time;

    double simple_interest = (principal * rate * time) / 100.0;

    double compound_interest = principal * pow((1 + rate / 100.0), time) - principal;

    cout << "Simple Interest: " << simple_interest << endl;

    cout << "Compound Interest: " << compound_interest << endl;

    return 0;

}
```

OUTPUT-



The screenshot shows a VS Code terminal window with the following tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The command prompt shows the execution of a C++ program. The user enters the principal amount as 10000, the annual interest rate as 5, and the time in years as 3. The program outputs the Simple Interest as 1500 and the Compound Interest as 1576.25.

```
PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
Enter principal amount: 10000
Enter annual interest rate (in %): 5
Enter time in years: 3
Simple Interest: 1500
Compound Interest: 1576.25
PS C:\Users\LENOVO\Desktop\Insiya\.vscode>
```

5)PROGRAM TO COMPUTE nPr and nCr

```
#include <iostream>

using namespace std;

long long factorial(int n) {

    long long fact = 1;

    for (int i = 2; i <= n; i++)

        fact *= i;

    return fact;

}

int main() {

    int n, r;

    cout << "Enter value of n: ";

    cin >> n;

    cout << "Enter value of r: ";

    cin >> r;

    if (r > n || n < 0 || r < 0) {

        cout << "Invalid input. Ensure that 0 <= r <= n." << endl;

        return 0;

    }

    long long npr = factorial(n) / factorial(n - r);

    long long ncr = factorial(n) / (factorial(r) * factorial(n - r));

    cout << "nPr: " << npr << endl;

    cout << "nCr: " << ncr << endl;

    return 0;

}
```

OUTPUT-

```
--
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
Enter value of n: 10
Enter value of r: 3
nPr: 720
nCr: 120
PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```


6) BISECTION METHOD

```
#include <iostream>

#include <cmath>

#include <iomanip>

using namespace std;

double f(double x) {

    return pow(x, 3) - 1;

}

int main() {

    double a, b, c, error;

    int maxIterations;

    cout << "Enter interval a and b such that f(a) and f(b) have opposite signs:\n";

    cin >> a >> b;

    if (f(a) * f(b) >= 0) {

        cout << "Invalid interval. f(a) and f(b) must have opposite signs." << endl;

        return 1;

    }

    cout << "Enter allowed error: ";

    cin >> error;

    cout << "Enter maximum number of iterations: ";

    cin >> maxIterations;

    cout << fixed << setprecision(6);

    int iteration = 0;

    while ((b - a) >= error && iteration < maxIterations) {

        c = (a + b) / 2;

        cout << "Iteration " << iteration + 1 << ": c = " << c << ", f(c) = " << f(c) << endl;

        if (fabs(f(c)) <= error) {

            break;

        }

        if (f(c) * f(a) < 0)

            b = c;

        else

            a = c;

    }
```

```

        iteration++;

    }

    cout << "\nApproximate root: " << c << endl;

    cout << "Total iterations: " << iteration << endl;

    return 0;

}

```

OUTPUT-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter interval a and b such that f(a) and f(b) have opposite signs:
0.5 1.9
Enter allowed error: 0.001
Enter maximum number of iterations: 20
Iteration 1: c = 1.200000, f(c) = 0.728000
Iteration 2: c = 0.850000, f(c) = -0.385875
Iteration 3: c = 1.025000, f(c) = 0.076891
Iteration 4: c = 0.937500, f(c) = -0.176025
Iteration 5: c = 0.981250, f(c) = -0.055202
Iteration 6: c = 1.003125, f(c) = 0.009404
Iteration 7: c = 0.992187, f(c) = -0.023255
Iteration 8: c = 0.997656, f(c) = -0.007015
Iteration 9: c = 1.000391, f(c) = 0.001172
Iteration 10: c = 0.999023, f(c) = -0.002927
Iteration 11: c = 0.999707, f(c) = -0.000879

Approximate root: 0.999707
Total iterations: 10
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █

```

7) REGULA-FALSE METHOD

```
#include <iostream>

#include <cmath>

#include <iomanip>

using namespace std;

double f(double x) {

    return pow(x, 3) - x - 11;

}

int main() {

    double a, b, c, fa, fb, fc, error;

    int maxIterations;

    cout << "Enter interval a and b such that f(a) and f(b) have opposite signs:\n";

    cin >> a >> b;

    if (f(a) * f(b) >= 0) {

        cout << "Invalid interval. f(a) and f(b) must have opposite signs." << endl;

        return 1;

    }

    cout << "Enter allowed error: ";

    cin >> error;

    cout << "Enter maximum number of iterations: ";

    cin >> maxIterations;

    cout << fixed << setprecision(6);

    int iteration = 0;

    while (iteration < maxIterations) {

        fa = f(a);

        fb = f(b);

        c = (a * fb - b * fa) / (fb - fa);

        fc = f(c);

        cout << "Iteration " << iteration + 1 << ": c = " << c << ", f(c) = " << fc << endl;

        if (fabs(fc) < error) {

            break;

        }

    }
```

```

    if (fa * fc < 0)

        b = c;

    else

        a = c;

    iteration++;

}

cout << "\nApproximate root: " << c << endl;

cout << "Total iterations: " << iteration << endl;

return 0;

}

```

OUTPUT –

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

```

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter interval a and b such that f(a) and f(b) have opposite signs:
2 3
Enter allowed error: 0.001
Enter maximum number of iterations: 20
Iteration 1: c = 2.277778, f(c) = -1.460048
Iteration 2: c = 2.350701, f(c) = -0.361202
Iteration 3: c = 2.368254, f(c) = -0.085595
Iteration 4: c = 2.372387, f(c) = -0.020076
Iteration 5: c = 2.373354, f(c) = -0.004697
Iteration 6: c = 2.373581, f(c) = -0.001098
Iteration 7: c = 2.373634, f(c) = -0.000257

Approximate root: 2.373634
Total iterations: 6
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █

```

8) NEWTON RAPHSON METHOD

```
#include <iostream>

#include <cmath>

using namespace std;

float f(float x) {

    return cos(x) - x * exp(x);

}

float df(float x) {

    return -sin(x) - exp(x) - x * exp(x);

}

int main() {

    float a, b;

    cout << "Enter initial approximation of root: ";

    cin >> a;

    for (int i = 1; i <= 15; i++) {

        float fa = f(a);

        float dfa = df(a);

        if (dfa == 0) {

            cout << "Derivative is zero. Cannot continue." << endl;

            return -1;

        }

        b = a - fa / dfa;

        cout << "Iteration " << i << ": Approximation to the root is: " << b << endl;

        if (fabs((b - a) / b) < 0.0001) {

            cout << "\nFinal root is: " << b << endl;

            return 0;

        }

        a = b;
```

```
}

cout << "\nMethod did not converge in 15 iterations. Last approximation: " << b << endl;

return 0;

}
```

OUTPUT-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter initial approximation of root: 0.5
Iteration 1: Approximation to the root is: 0.518026
Iteration 2: Approximation to the root is: 0.517757
Iteration 3: Approximation to the root is: 0.517757

Final root is: 0.517757
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```

9) GAUSS ELIMINATION METHOD

```
#include <iostream>

#include <cmath>

using namespace std;

int main() {

    int n;

    float A[20][20], x[10], c, sum = 0.0;

    cout << "Enter the order of the matrix: ";

    cin >> n;

    cout << "\nEnter the elements of the augmented matrix row-wise:\n";

    for (int i = 1; i <= n; i++) {

        for (int j = 1; j <= n + 1; j++) {

            cout << "A[" << i << "][" << j << "]: ";

            cin >> A[i][j];

        }

    }

    for (int j = 1; j <= n; j++) {

        for (int i = 1; i <= n; i++) {

            if (i > j) {

                c = A[i][j] / A[j][j];

                for (int k = 1; k <= n + 1; k++) {

                    A[i][k] -= c * A[j][k];

                }

            }

        }

    }

    x[n] = A[n][n + 1] / A[n][n];

    for (int i = n - 1; i >= 1; i--) {

        sum = 0.0;

        for (int j = i + 1; j <= n; j++) {

            sum += A[i][j] * x[j];

        }

        x[i] = (A[i][n + 1] - sum) / A[i][i];

    }

}
```

```

}

cout << "\nThe solution is:\n";

for (int i = 1; i <= n; i++) {

    cout << "x[" << i << "] = " << x[i] << endl;

}

return 0;

}

```

OUTPUT-

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

```

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter the order of the matrix: 3

Enter the elements of the augmented matrix row-wise:
A[1][1]: 3
A[1][2]: 2
A[1][3]: -4
A[1][4]: 3
A[2][1]: 2
A[2][2]: 3
A[2][3]: 3
A[2][4]: 15
A[3][1]: 5
A[3][2]: -3
A[3][3]: 1
A[3][4]: 14

The solution is:
x[1] = 3
x[2] = 1
x[3] = 2
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █

```


10) GAUSS-JORDAN METHOD

```
#include <iostream>

#include <iomanip>

using namespace std;

const int SIZE = 10;

int main() {

    int n;

    float a[SIZE][SIZE + 1], x[SIZE];

    cout << "Enter the number of unknowns: ";

    cin >> n;

    cout << "Enter the augmented matrix (row-wise):\n";

    for (int i = 0; i < n; i++) {

        for (int j = 0; j <= n; j++) {

            cin >> a[i][j];

        }

    }

    for (int i = 0; i < n; i++) {

        float diag = a[i][i];

        if (diag == 0) {

            cout << "Mathematical Error: Division by zero.\n";

            return -1;

        }

        for (int j = 0; j <= n; j++) {

            a[i][j] /= diag;

        }

        for (int k = 0; k < n; k++) {

            if (k != i) {

                float factor = a[k][i];

                for (int j = 0; j <= n; j++) {

                    a[k][j] -= factor * a[i][j];

                }

            }

        }

    }

}
```

```

}

for (int i = 0; i < n; i++) {

    x[i] = a[i][n];

}

cout << "\nSolution:\n";

for (int i = 0; i < n; i++) {

    cout << "x[" << i + 1 << "] = " << fixed << setprecision(3) << x[i] << endl;

}

return 0;

}

```

OUTPUT –

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

```

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter the number of unknowns: 3
Enter the augmented matrix (row-wise):
3 2 -4 3
2 3 3 15
5 -3 1 14

Solution:
x[1] = 3.000
x[2] = 1.000
x[3] = 2.000
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> 

```

11) NEWTON'S FORWARD INTERPOLATION FORMULA

```
#include <iostream>

#include <iomanip>

using namespace std;

int main() {

    int n, i, j;

    float x[10], y[10][10], h, p, xp, yp;

    cout << "Enter number of data points: ";

    cin >> n;

    cout << "Enter the values of x:\n";

    for (i = 0; i < n; i++) {

        cin >> x[i];

    }

    cout << "Enter the values of y:\n";

    for (i = 0; i < n; i++) {

        cin >> y[i][0];

    }

    cout << "Enter the value of x for which you want to find y: ";

    cin >> xp;

    h = x[1] - x[0];

    p = (xp - x[0]) / h;

    for (j = 1; j < n; j++) {

        for (i = 0; i < n - j; i++) {

            y[i][j] = y[i + 1][j - 1] - y[i][j - 1];

        }

    }

    yp = y[0][0];

    float term = 1;

    for (i = 1; i < n; i++) {

        term *= (p - (i - 1)) / i;

        yp += term * y[0][i];

    }
```

```
cout << fixed << setprecision(4);
```

```
cout << "Interpolated value of y at x = " << xp << " is " << yp << endl;
```

```
return 0;
```

OUTPUT-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if (\$?) { g++ NSCP.cpp -o NSCP } ; if (\$?) { .\NSCP }
Enter number of data points: 5
Enter the values of x:
0 1 2 3 4
Enter the values of y:
1 3 7 13 21
Enter the value of x for which you want to find y: 2.5
Interpolated value of y at x = 2.5000 is 9.7500
- PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █

12) NEWTON'S BACKWARD INTERPOLATION FORMULA

```
#include <iostream>

using namespace std;

int fact(int n) {

    int f = 1;

    for (int i = 2; i <= n; i++)

        f *= i;

    return f;

}

int main() {

    int n;

    float x[10], y[10][10], xp, h, p, yp;

    cout << "Enter number of data points: ";

    cin >> n;

    cout << "Enter the values of x:\n";

    for (int i = 0; i < n; i++) {

        cin >> x[i];

    }

    cout << "Enter the values of y:\n";

    for (int i = 0; i < n; i++) {

        cin >> y[i][0];

    }

    cout << "Enter the value of x for which you want to find y: ";

    cin >> xp;

    for (int j = 1; j < n; j++) {

        for (int i = n - 1; i >= j; i--) {

            y[i][j] = y[i][j-1] - y[i-1][j-1];

        }

    }

    h = x[1] - x[0];

    p = (xp - x[n-1]) / h;

    yp = y[n-1][0];
```

```

for (int i = 1; i < n; i++) {

    float p_term = 1;

    for (int j = 0; j < i; j++)

        p_term *= (p + j);

    yp += (p_term * y[n-1][i]) / fact(i);

}

cout << "Interpolated value at x = " << xp << " is y = " << yp << endl;

return 0;

}

```

OUTPUT-

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

```

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
Enter number of data points: 4
Enter the values of x:
5 6 7 8
Enter the values of y:
150 392 1452 2368
Enter the value of x for which you want to find y: 7.5
Interpolated value at x = 7.5 is y = 1988.12
PS C:\Users\LENOVO\Desktop\Insiya\.vscode>

```

13) SIMPSON'S THREE - EIGHT RULE

```
#include <iostream>

#include <cmath>

using namespace std;

double f(double x) {

    return 1 / (1 + x * x); // Function to integrate

}

int main() {

    double a, b, h, sum;

    int n;

    cout << "Enter lower limit: ";

    cin >> a;

    cout << "Enter upper limit: ";

    cin >> b;

    cout << "Enter number of sub-intervals (must be multiple of 3): ";

    cin >> n;

    h = (b - a) / n;

    sum = f(a) + f(b);

    for (int i = 1; i < n; i++) {

        if (i % 3 == 0)

            sum += 2 * f(a + i * h);

        else

            sum += 3 * f(a + i * h);

    }

    sum *= (3 * h / 8);

    cout << "Approximate value of integral: " << sum << endl;

    return 0;

}
```

OUTPUT –

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter lower limit: 0
  Enter upper limit: 1
  Enter number of sub-intervals (must be multiple of 3): 6
  Approximate value of integral: 0.785396
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```

14) TRAPEZOIDAL RULE

```
#include <iostream>

#include <cmath>

using namespace std;

double f(double x) {

    return 1 / (1 + pow(x, 2));

}

int main() {

    double lower, upper, integration = 0.0, stepSize, k;

    int i, subInterval;

    cout << "Enter lower limit of integration: ";

    cin >> lower;

    cout << "Enter upper limit of integration: ";

    cin >> upper;

    cout << "Enter number of sub intervals: ";

    cin >> subInterval;

    stepSize = (upper - lower) / subInterval;

    integration = f(lower) + f(upper);

    for (i = 1; i < subInterval; i++) {

        k = lower + i * stepSize;

        integration += 2 * f(k);

    }

    integration = integration * stepSize / 2;

    cout << "\nRequired value of integration is: " << fixed << integration << endl;

    return 0;

}
```

OUTPUT-

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
----------	--------	---------------	----------	-------

```
PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if ($?) { g++ NSCP.cpp -o NSCP } ; if ($?) { .\NSCP }
● Enter lower limit of integration: 2
Enter upper limit of integration: 3
Enter number of sub intervals: 5

Required value of integration is: 0.142230
○ PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █
```


15) RUNGE KUTTA METHOD

```
#include <iostream>

#include <cmath>

using namespace std;

double f(double x, double y) {

    return x * x + y * y;

}

double rungeKutta(double x0, double y0, double x, double h) {

    double y = y0;

    double n = (x - x0) / h;

    for (int i = 0; i < n; ++i) {

        double k1 = h * f(x0, y);

        double k2 = h * f(x0 + 0.5 * h, y + 0.5 * k1);

        double k3 = h * f(x0 + 0.5 * h, y + 0.5 * k2);

        double k4 = h * f(x0 + h, y + k3);

        y = y + (1.0 / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4);

        x0 += h;

    }

    return y;

}

int main() {

    double x0, y0, x, h;

    cout << "Enter the initial value of x (x0): ";

    cin >> x0;

    cout << "Enter the initial value of y (y0): ";

    cin >> y0;

    cout << "Enter the value of x where you want to calculate y: ";

    cin >> x;

    cout << "Enter the step size (h): ";

    cin >> h;
```

```
double result = rungeKutta(x0, y0, x, h);

cout << "The value of y at x = " << x << " is: " << result << endl;

return 0;

}
```

OUTPUT –

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Insiya> cd "c:\Users\LENOVO\Desktop\Insiya\.vscode\" ; if (\$?) { g++ NSCP.cpp -o NSCP } ; if (\$?) { .\NSCP }

- Enter the initial value of x (x0): 1.0
Enter the initial value of y (y0): 0.0
Enter the value of x where you want to calculate y: 1.4
Enter the step size (h): 0.1
The value of y at x = 1.4 is: 0.622698
- PS C:\Users\LENOVO\Desktop\Insiya\.vscode> █

